



**FEU INSTITUTE OF TECHNOLOGY**

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

# **CS0065 (INTELLIGENT SYSTEMS)**

## **TECHNICAL ASSESSMENT**

# **4**

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED  
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

|                     |                         |
|---------------------|-------------------------|
| <b>Student Name</b> | KARL GABRIEL R. JARILLA |
| <b>Section:</b>     | AN42                    |
| <b>Professor:</b>   | MS. CRISOLA TAN         |
| <b>Date:</b>        | October 3, 2026         |

## I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

### 1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

### 2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

## II. TECHNICAL ASSESSMENT

### IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

#### *Objective*

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

#### **Materials Needed**

##### 1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

##### 2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

##### 3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

#### **General Instructions**

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4\_lastname1.pdf**  
(Example: TA2\_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

## LABORATORY TASKS

### PART 1 — Knowledge Representation (KR) Setup

#### Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

**Example structure to follow:**

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

### PART 2 — Implement Rule-Based Reasoning (RBR)

#### Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

**Example:**

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

#### Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

#### Task 2C — Display Output

---

**Example [Program should print:]**

Rule-Based Reasoning Actions:

- Turn on the aircon
  - Bring umbrella
- 

### **PART 3 — Implement Case-Based Reasoning (CBR)**

#### **Task 3A — Build a Case Base**

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

---

**Example:**

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

---

#### **Task 3C — Reuse Step**

Return the solution of the most similar case.

#### **Task 3D — Revise Step**

Allow student to adjust solution manually or return unchanged.

#### **Task 3E — Retain Step**

Add the new case to the case base.

### **PART 4 — Comparative Reflection**

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
  2. Which is more flexible in solving unstructured problems?
  3. Which is better for situations with continuously changing data?
  4. Examples of real-world applications for both.
- 
- Between the two systems, I found Rule-Based Reasoning (RBR) much simpler to code since it just uses basic IF-THEN logic to evaluate specific, hardcoded conditions. On the other hand, Case-Based Reasoning (CBR) offers far more flexibility for unstructured problems because it doesn't require an exact match; instead, it calculates similarity scores to find the closest past experience. This makes CBR much better suited for environments where data is always changing, as its "Retain" phase lets the system learn and expand its database dynamically without needing any manual code updates. Looking at real-world applications, RBR is great for strict, rule-heavy tasks like ad hoc network routing protocols or basic hardware temperature monitoring. Meanwhile, CBR is perfect for dynamic, unpredictable scenarios, much like intelligent traffic management systems or diagnosing vehicle issues, where past data directly helps solve new, unique problems.

## Your Code - Laboratory Tasks ↓↓

# Task 1A: Create a Knowledge Base

# Use Case: PC Gaming & Network Performance

```
knowledge_base = {
    "categories": ["network_status", "hardware_status", "game_performance"],
    "facts": {
        "ping_ms": 135,
        "fps": 45,
        "packet_loss": True,
        "cpu_temp_c": 86,
        "gpu_temp_c": 75
    }
}
```

# Task 2A: Create at least 3 IF THEN rules

def rule\_high\_ping(facts):

# Rule 1: Triggers if network latency is high or unstable

if facts["ping\_ms"] > 100 or facts["packet\_loss"]:

return "Restart your router and check the ethernet cable."

return None

def rule\_low\_fps(facts):

# Rule 2: Triggers if frame rate is below competitive standards

if facts["fps"] < 60:

return "Lower Valorant graphics settings and disable background apps."

return None

def rule\_overheating(facts):

# Rule 3: Triggers if hardware temperatures are dangerous

if facts["cpu\_temp\_c"] > 80 or facts["gpu\_temp\_c"] > 80:

return "Check PC cooling and clean dust from fans."

return None

rules = [rule\_high\_ping, rule\_low\_fps, rule\_overheating]

# Task 2B: Implement an Inference Engine

def inference\_engine(kb, rule\_list):

actions = []

facts = kb["facts"]

# Reads the knowledge base and checks each rule

for rule in rule\_list:

action = rule(facts)

```

    if action:
        actions.append(action) # Returns all actions inferred
    return actions

# Task 2C: Display Output
print("--- RBR System Output ---")
print("Rule-Based Reasoning Actions:")
rbr_actions = inference_engine(knowledge_base, rules)
for action in rbr_actions:
    print(f"- {action}")
print("\n")

# Task 3A: Build a Case Base
# Use Case: Vehicle Maintenance
case_base = [
    {
        "problem": {"engine_wont_start": True, "headlights_dim": True, "clicking_noise": True},
        "solution": "Replace or jump-start the vehicle battery."
    },
    {
        "problem": {"engine_wont_start": True, "headlights_dim": False, "clicking_noise": False},
        "solution": "Check the starter motor or fuel pump."
    },
    {
        "problem": {"engine_wont_start": False, "steering_vibrates": True, "check_engine": False},
        "solution": "Check tire alignment and wheel balance."
    }
]

# Similarity function to support the Retrieve step
def calculate_similarity(new_case, past_case):
    matches = 0
    total = len(new_case)
    for key in new_case:
        if key in past_case and new_case[key] == past_case[key]:
            matches += 1
    return matches / total if total > 0 else 0

# Task 3C: Reuse Step (Includes Retrieve logic)
def retrieve_and_reuse(new_problem, cb):
    best_match = None
    highest_score = -1

    for case in cb:

```

```

    score = calculate_similarity(new_problem, case["problem"])
    if score > highest_score:
        highest_score = score
        best_match = case

    print(f"[Retrieve] Highest similarity score: {highest_score * 100:.2f}%")
    print(f"[Reuse] Suggested Solution: {best_match['solution']}")
    return best_match['solution'] # Returns the solution of the most similar case

# Task 3D: Revise Step
def revise(suggested_solution):
    # Simulating a user prompt adjusting the solution manually
    print("[Revise] Evaluating solution...")
    user_override = "" # If a user typed a string here, it would override the solution
    if user_override:
        print(f"Solution revised to: {user_override}")
        return user_override
    print("Solution accepted unchanged.")
    return suggested_solution

# Task 3E: Retain Step
def retain(new_problem, final_solution, cb):
    new_case = {"problem": new_problem, "solution": final_solution}
    cb.append(new_case) # Adds the new case to the case base
    print("[Retain] New case successfully added to the Case Base.")

# CBR Execution
print("--- CBR System Output ---")
# Simulating a new unstructured problem (e.g., Honda BR-V battery issue)
new_vehicle_problem = {"engine_wont_start": True, "headlights_dim": True, "clicking_noise": False}
suggested = retrieve_and_reuse(new_vehicle_problem, case_base)
final = revise(suggested)
retain(new_vehicle_problem, final, case_base)

```